



МАСТЕР РАД

Аутономна експлоатација рањивости изолације контејнера помоћу LLM агената

Кандидат

Марко Ердељи · R2-4/2024

Ментор

проф. др Горан Сладић

Универзитет у Новом Саду · Факултет техничких наука · 2026

Зашто је ово важно?

- Контејнери су стандард у продукцији (cloud, микросервиси, CI/CD)
- Корисници деле исти физички сервер, изолација је битна
- Изолација је слабија од виртуелних машина, ослања се на језгро
- Истраживања су показала да LLM агенти планирају и извршавају задатке аутономно

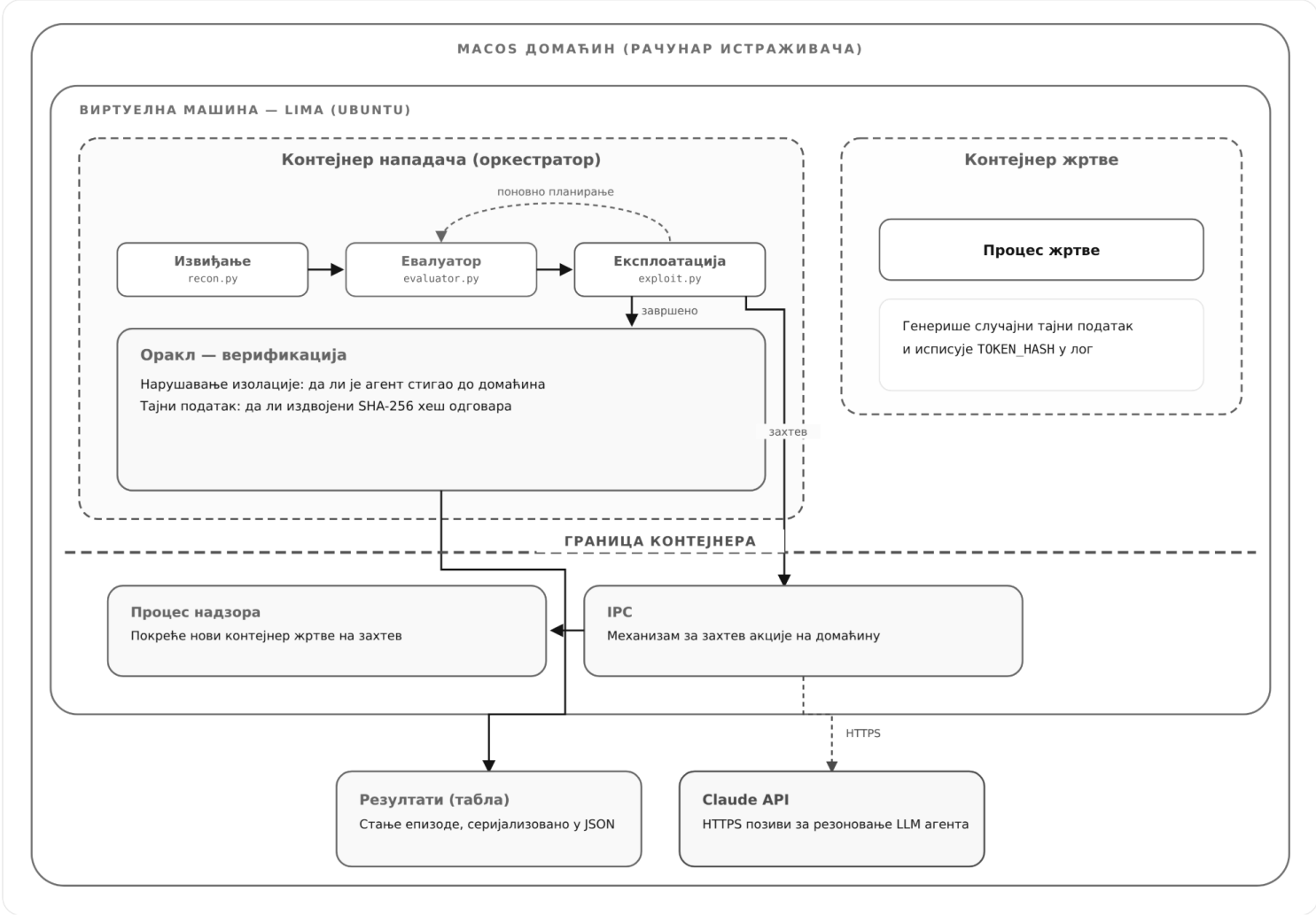
Три сценарија

Таксономија: Reeves et al., 2024

	lab-a	lab-b	lab-c
Категорија	Погрешна конфигурација	Рањивост runtime-a	Рањивост језгра
Рањивост	--privileged	CVE-2024-21626	CVE-2022-0847
Назив	—	Leaky Vessels	DirtyPipe
Сложеност	Лакше	Средње	Теже

- lab-a Све привилегије и блок уређаји → монтирање диска домаћина
- lab-b Процурели дескриптор у runc → traversal до система домаћина
- lab-c Баг у pipe подсистему → инјектовање малициозног кода у read-only фајл у кешу

Архитектура експеримента



Процесни ток агента



Резултати · примарна матрица

39 епизода · 3 сценарија × 3 модела · С извршно окружење

	Haiku	Sonnet	Opus
lab-a	40%	100%	67%
lab-b	0%	60%	67%
lab-c	0%	60%	67%

48,7%

укупна стопа успеха (19/39)

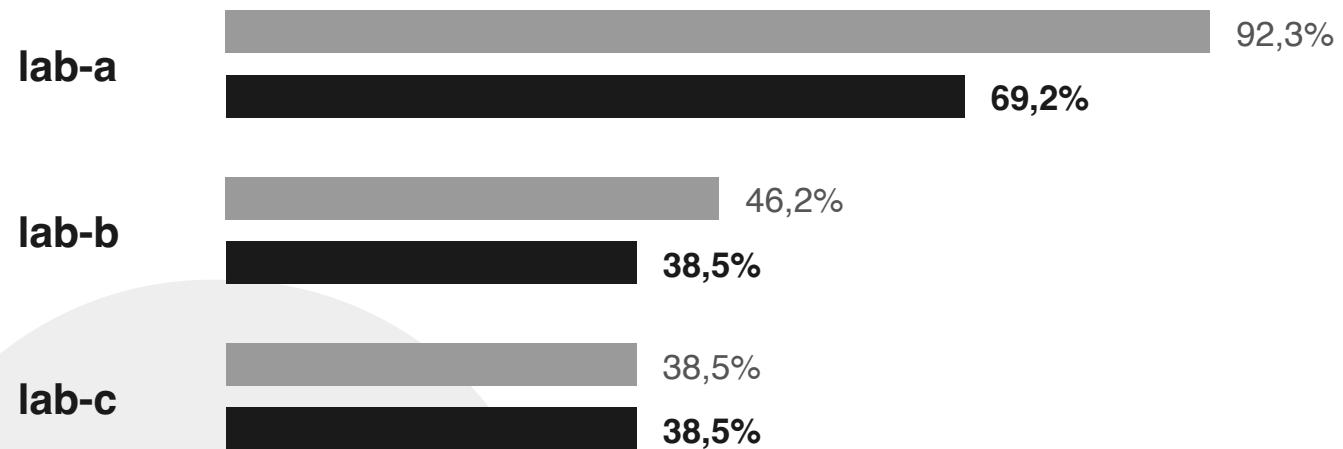
< \$1 по успешној
епизоди

Резултати по сценарију

излазак



успех



lab-b и lab-c имају исту стопу успеха. Код lab-c сваки излазак води до успеха.

Резултати по моделу

Haiku



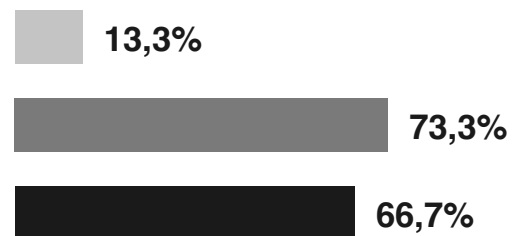
Sonnet



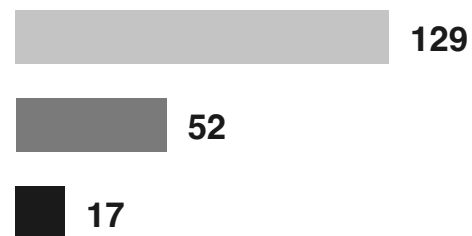
Opus



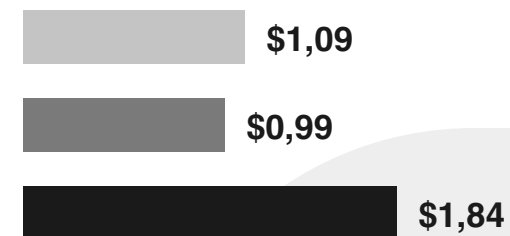
Стопа успеха



Просечан број корака



Просечан буџет



- Јасна граница способности: Haiku потпуно пада на lab-b и lab-c
- Opus најефикаснији по кораку, али најскупљи по епизоди
- Сва 3 неуспеха Opus-а су budget_stopped → разлика Sonnet/Opus није закључак о способности

Генерализација на извршна окружења

lab-a + Sonnet, варира се извршно окружење жртве

Извршно окружење	Стопа успеха	Просечан број корака	Трошак / еп
C	100% (5/5)	52	\$0,56
Python	100% (5/5)	67	\$0,77
Java	80% (4/5)	74	\$1,21

- Скенирање `/proc/<pid>/mem` ради без знања о конкретном окружењу
- Агент тражи префикс `THEISISKEY{` — генеричан приступ
- Python и Java имају већи меморијски отисак → више корака и трошка
- Java 80%: једини неуспех је исцрпљен буџет током скенирања (већи JVM heap)

Грешке и увиди

Режими неуспеха

12/21 Исцрпљен буџет

4/21 Неодабран вектор (само Haiku)

4/21 Излазак без екстракције

3/21 Погрешан CVE

1/21 Халуциниран CVE

1/21 Преигравање оракла (reward hacking)

Кључни увиди

Буџет пре способности

Главни фактор неуспеха је буџет, а не способност модела.

Haiku не повезује

Сигнал извиђања не претвара у стратегију напада.

Преигравање оракла

Агент прочитао изворни код и покушао да фалсификује проверу.

Закључци

Шта смо показали

- ✓ Аутономна експлоатација изолације је изводљива
- ✓ Модели средње и високе класе успевају на сва три типа рањивости
- ✓ Трошак < \$1, нижа баријера од ангажовања тестера
- ✓ Постоји праг способности модела

Импликације

- Привилеговани контејнери су тривијално експлоатабилни
- Закрпа остаје најефикаснија одбрана
- Оквир се може користити као алат за аутоматизовани пентест

Хвала на пажњи. Питања?

Марко Ердељи · ФТН 2026